



Intro to Containers & Docker

SECTU

Daniel Fajardo

What is a dependency?

Third-party libraries,
configuration,
environment variables,
etc.

Runtime

System & OS

OS kernel, system libraries
(glibc, libssl, etc.), system
utilities (time zones,
certificates)

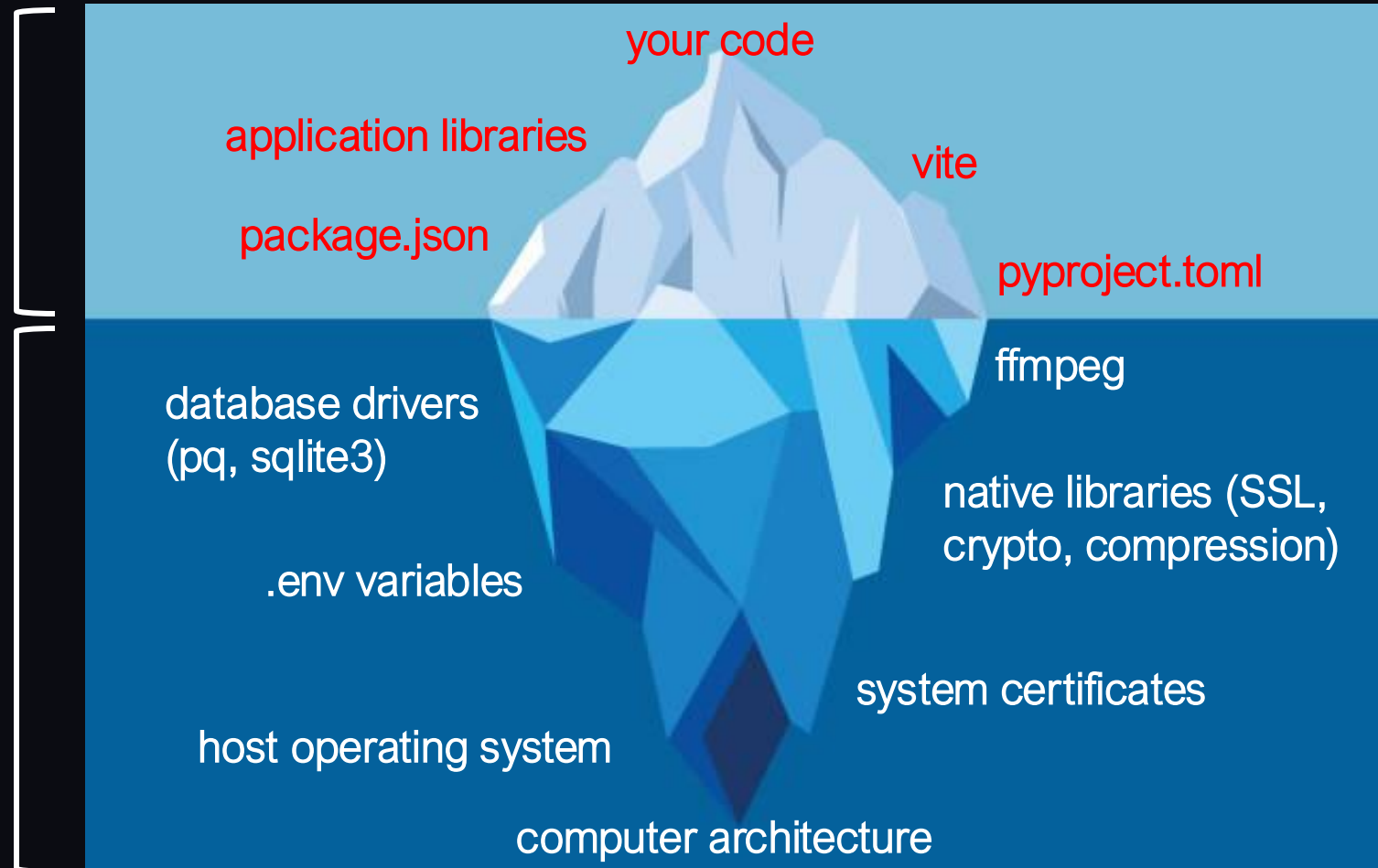
CPU architecture,
specialized computing
hardware

Hardware

The dependency iceberg

the dependencies you
think about

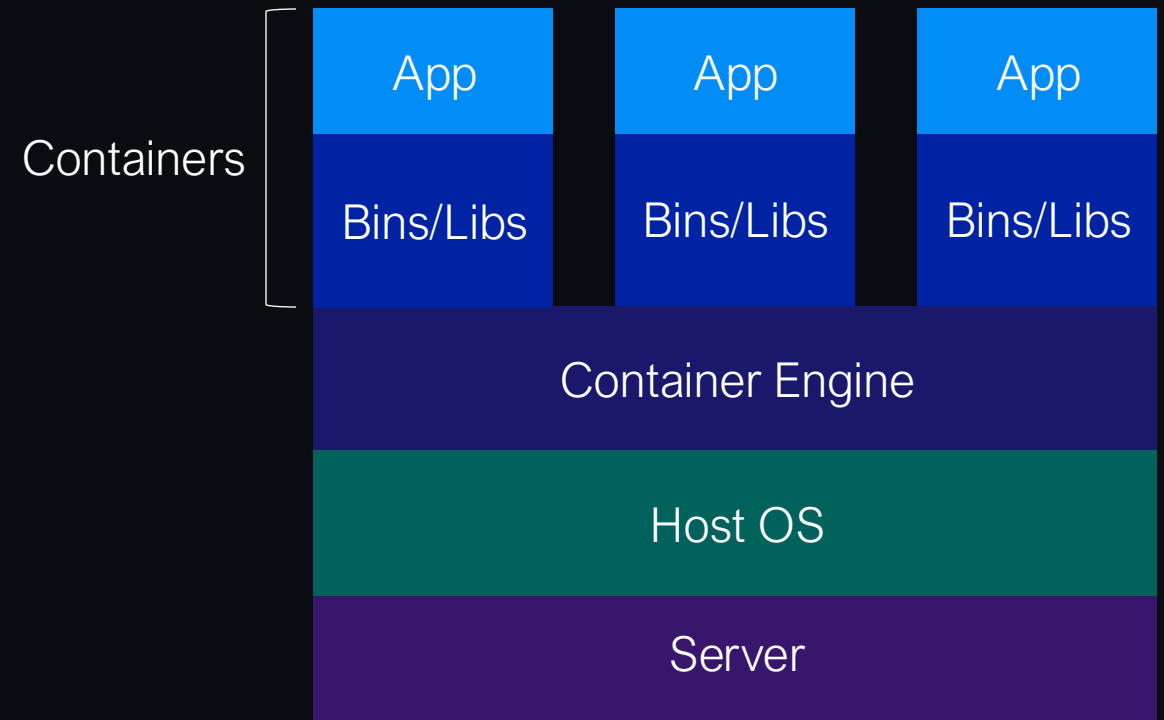
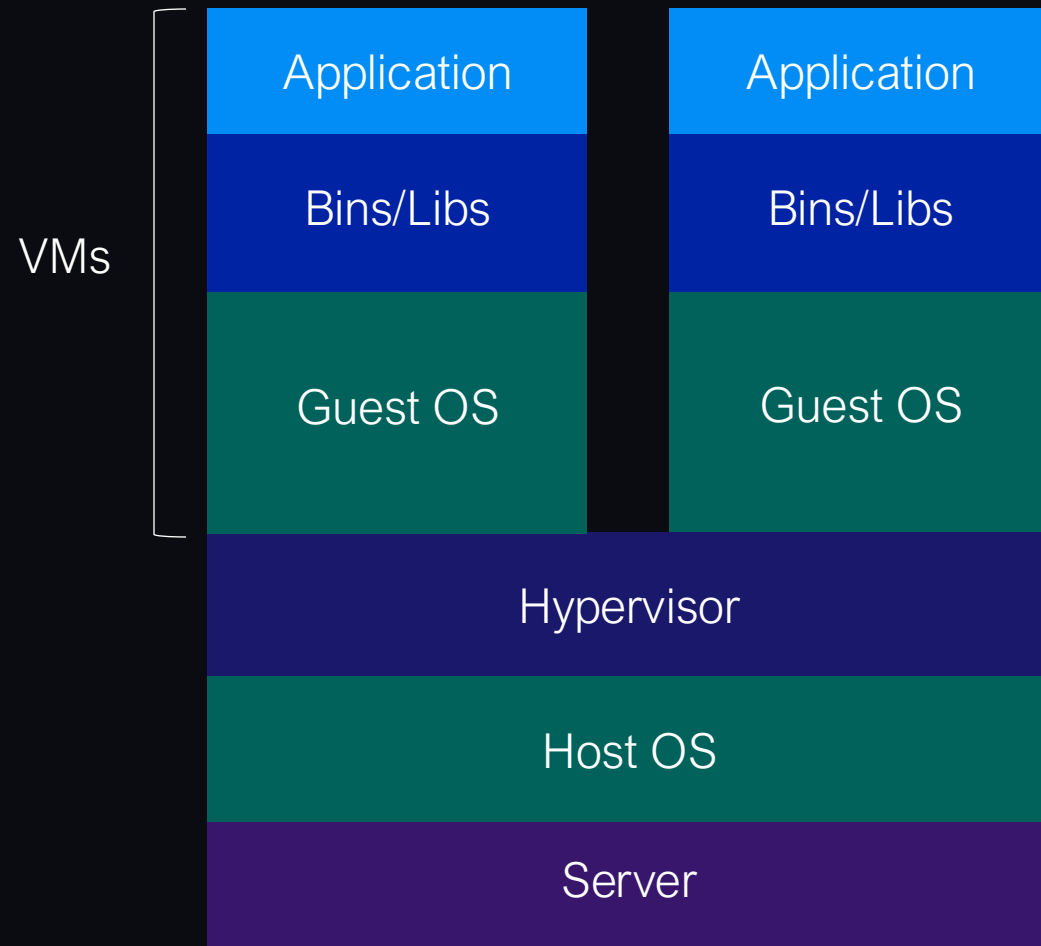
everything else



Container philosophy

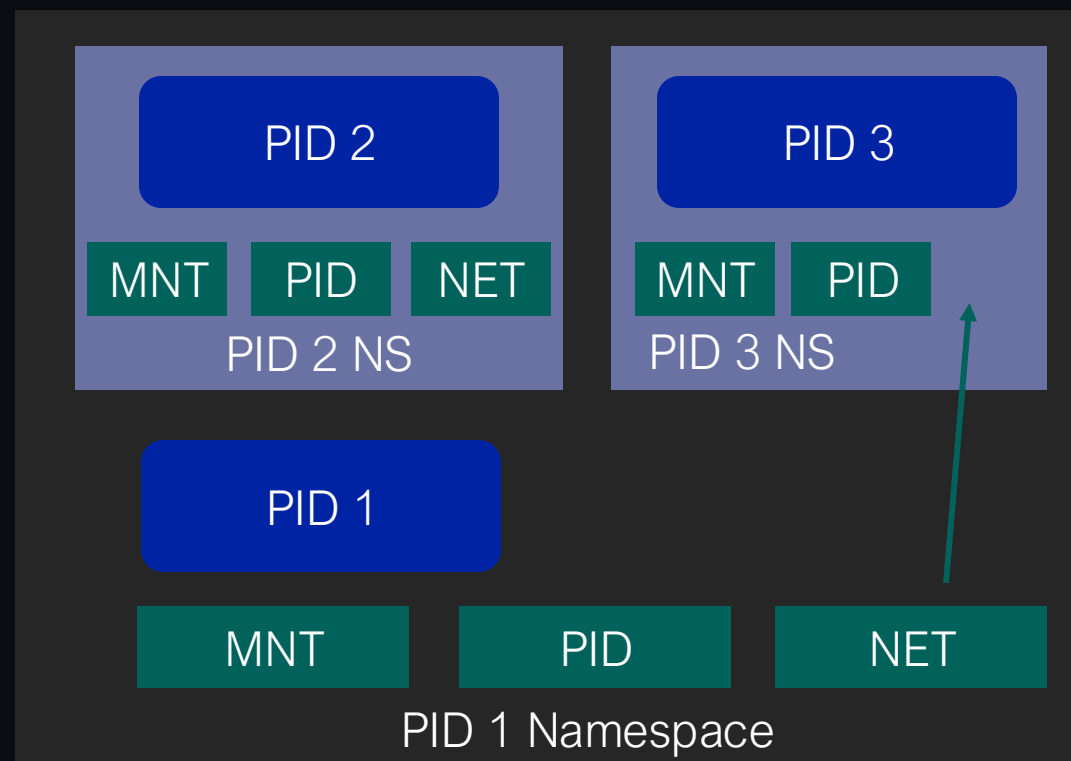
- Ship the *entire environment* for the application
 - Application files/binaries
 - System libraries and utilities
- Treat dependencies as explicit and immutable
 - No dependency should be unknown
 - No dependency should change without us knowing

Virtual Machines vs. Containers



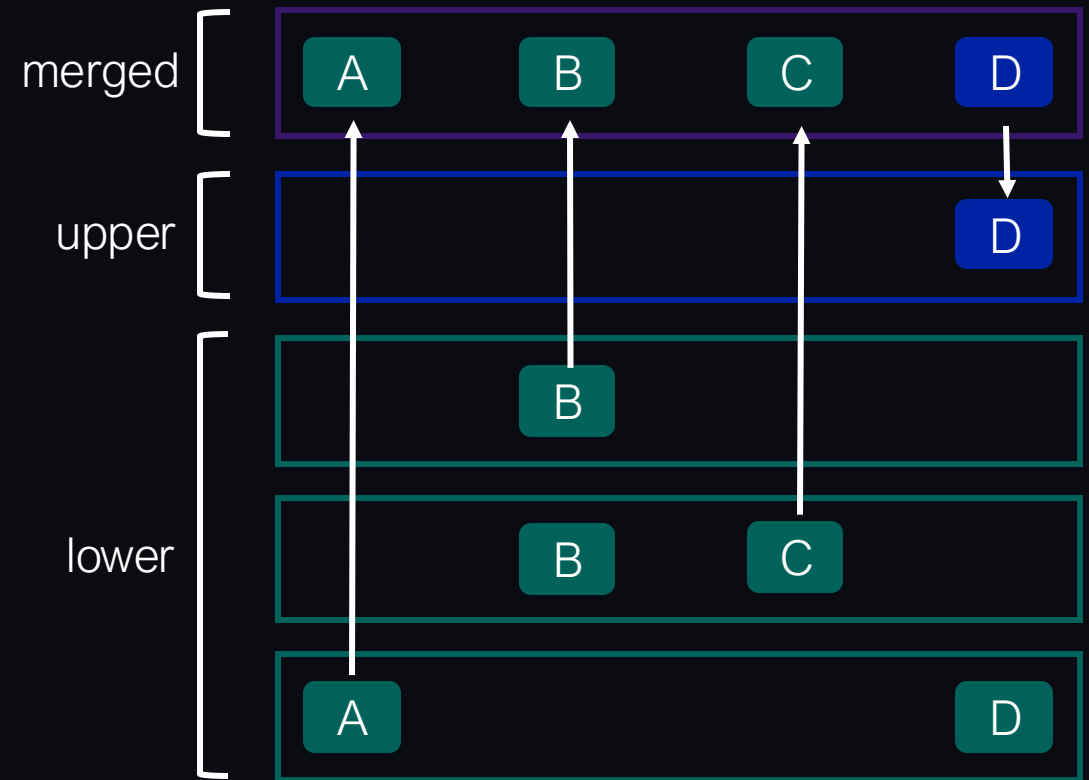
How containers work – namespaces, cgroups, and pivot_root

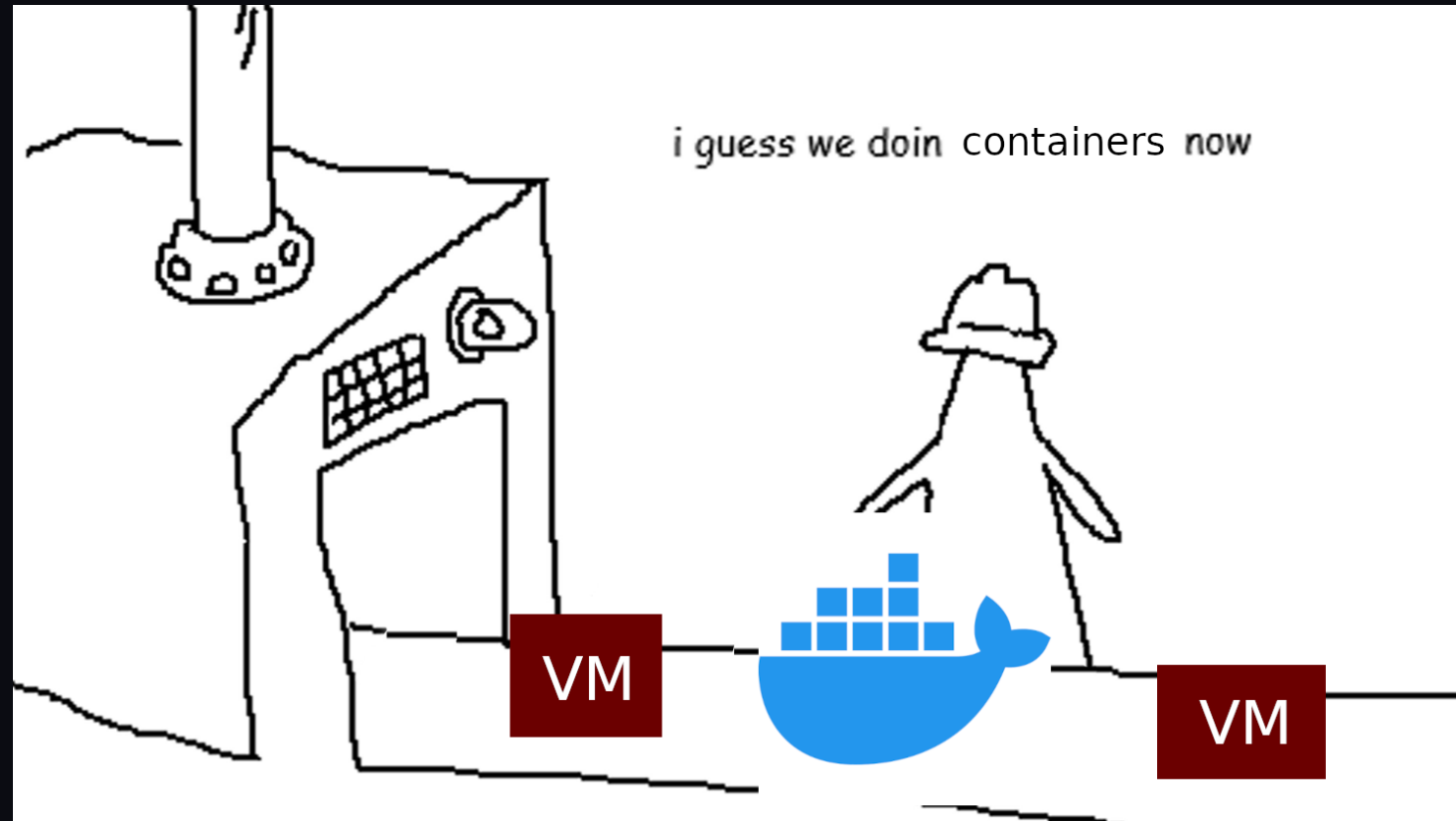
- **cgroups** – restraining system resources for a process
 - CPU throttling
 - Memory limits
- **namespaces** – what your process can see
 - MNT – filesystem mount points
 - PID – process counter & tree (hierarchical)
 - NET – what networks are available
- **pivot_root** – change a process's filesystem root
 - Prevents container from seeing/interacting with host
 - Applies to the MNT namespace of the process
 - chroot applies only to the current process



How containers work – union file systems (UFS) and OverlayFS

- Union file systems represent multiple "stacked" directories as a single file system
- OverlayFS is a Linux file system implementation for UFS
 - "lower-dir" - read only, bottom of stack
 - "upper-dir" - read-write
 - "merged" - view of unioned filesystem





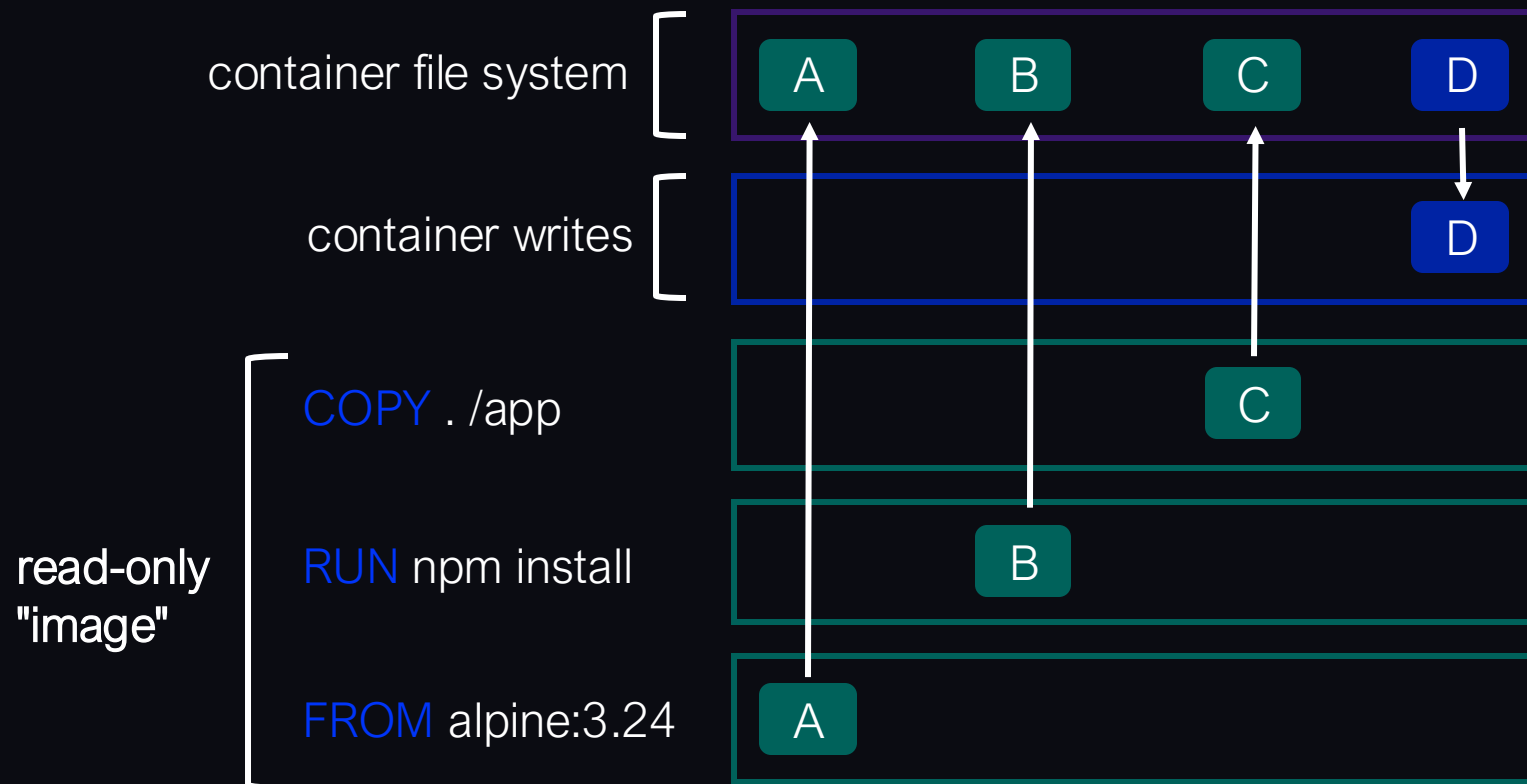
What is Docker?

Docker provides an easy UI/UX for managing containers and the Linux primitives around them



Layers, Images, and Containers

- "Layer" = change in filesystem
- "Image" = stack of read-only layers
- Docker represents the process of adding new layers using a [Dockerfile](#)



Anatomy of a Dockerfile

oven/bun is called the "base image"

Multiple bases -> "multi-stage" build

- Only the *final* stage is distributed for deployments, meaning you can make very small image sizes

```
FROM oven/bun:1.3.13 AS builder

WORKDIR /app

COPY package.json package-lock.json ./
RUN bun install --frozen-lockfile

COPY . .
RUN bun run build

FROM nginx:alpine AS runtime

WORKDIR /usr/share/nginx/html
RUN rm -rf ./*
COPY --from=builder /app/dist .

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Unchanged layers can be cached; good practice to treat package dependencies separately, since they change less than our code does

"EXPOSE" is just for documentation, so ops teams know which port to use

"CMD" specifies the default arguments for the container's process

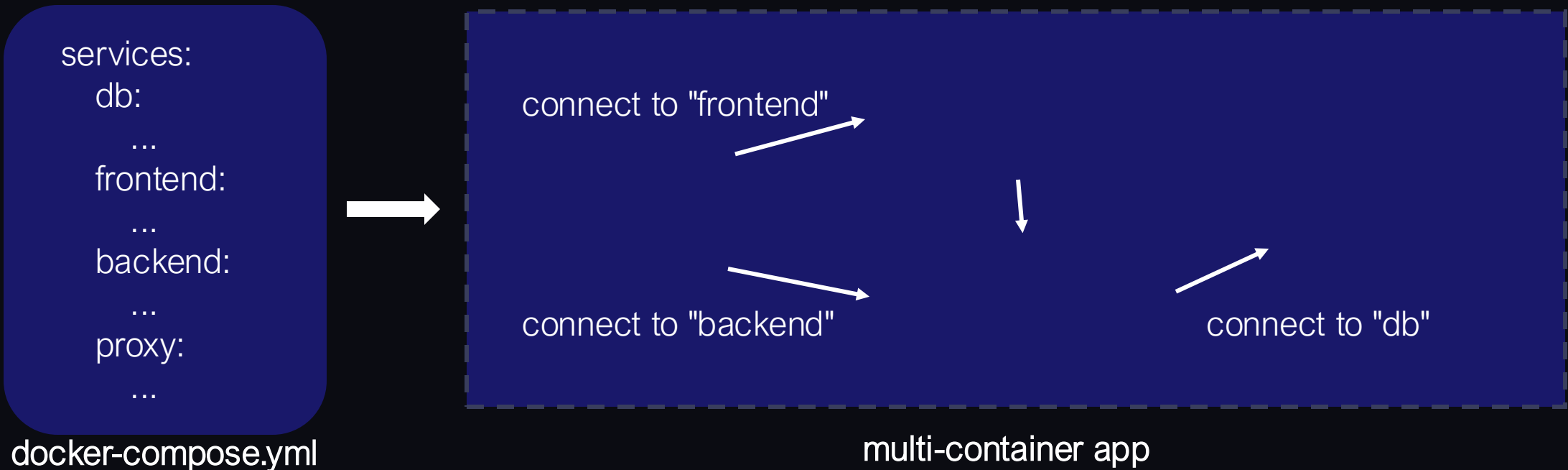
Demo: Containerize a Go backend

What if my application has multiple services?

- Multi-service dependencies
 - Databases
 - Producer-consumer models
 - Message Queues
 - Caches
- Tedious to manage all of this using Docker Desktop or Docker CLI

Docker Compose

- Declarative YAML configuration for many services
- Automatic hostname resolution (service "db" can now be connected to at host "db")
- Easy manage using Docker CLI



Demo: Orchestrating a local app with Docker Compose

Summary

- Dependencies come in many forms
- Containers are just *isolated processes*
 - Forces an app's dependencies to be *explicit*
- Docker is a UI/UX abstraction over container management for developers
- Create an image using a Dockerfile
- Orchestrate multi-service apps using Docker Compose

Q/A & Resources

- Demos, slides on GitHub: <https://github.com/TU-Software/meetings/>
- Installing Docker on Windows: <https://docs.docker.com/desktop/setup/install/windows-install/>
- Installing Docker on MacOS: <https://docs.docker.com/desktop/setup/install/mac-install/>
- Docker documentation: <https://docs.docker.com/>
- Docker Compose documentation: <https://docs.docker.com/compose/>

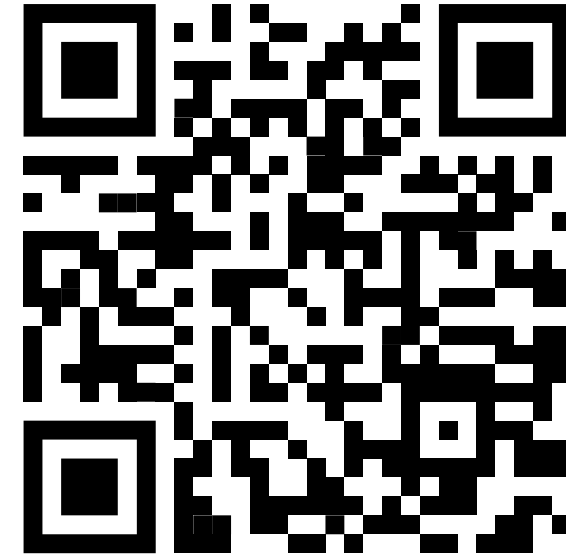
SECTU Links

[Join the discord](#)



SCAN ME

[Membership Roster](#)



SCAN ME